A Control and Embedded Software System for a Bioinspired Hummingbird Flapping-Wing Micro Aerial Vehicle

Nezih Arhan Tözenbilek
Department of Computer Engineering
Hacettepe University
Ankara, Türkiye
Advisor: Assoc. Prof. Dr. Harun Artuner

***Abstract*—This paper presents the design, implementation, and validation of the control and embedded software sub-system of a bioinspired hummingbird flapping-wing micro aerial vehicle (FWMAV). The system pairs an ESP32-C3 RISC-V microcontroller running custom firmware with a Jetpack Compose Android application, linked over a custom Bluetooth Low Energy (BLE) Generic Attribute (GATT) profile. The firmware drives a DRV8833 H-bridge in a sign-magnitude PWM scheme at $20\,\mathrm{kHz}$, exposes a 9-byte command and 20-byte telemetry channel protected by an 8-bit cyclic redundancy check, and enforces three independent fail-safes: a $500\,\mathrm{ms}$ command watchdog, an emergency stop on link loss, and a CRC-driven emergency stop on malformed packets. A complementary filter ($\alpha = 0.98$) and anti-windup proportional–integral–derivative (PID) controllers for roll and pitch are implemented and validated against a synthetic IMU model in a host-side regression test and on the embedded target under QEMU; physical sensor integration (MPU-6050) is one of three named integration tasks left for hardware bring-up. The mechanical airframe sub-system, developed by a separate team, is out of scope for this work; the software pipeline—phone to motor—was demonstrated on physical hardware with a brushed DC motor driven through the entire chain from the Android application over BLE.**

***Index Terms*—Embedded systems, Bluetooth Low Energy, ESP32-C3, attitude estimation, complementary filter, PID control, flapping-wing micro aerial vehicle, bioinspired robotics, Android, real-time safety.**

I. Introduction

Bioinspired flapping-wing micro aerial vehicles (FWMAVs) offer hovering and agility in regimes where fixed-wing and rotary-wing platforms struggle. Among biological models, the hummingbird is notable: its symmetric wing stroke and rapid wingbeat support hovering, lateral translation, and reverse flight [1], [2], [7]. Building such a platform demands tight integration of aerodynamic, mechanical, electronic, and software sub-systems.

This paper addresses the *control and embedded software* sub-system of a hummingbird-inspired FWMAV. The system comprises: (i) firmware for an ESP32-C3 microcontroller that drives the propulsion actuator through a DRV8833 dual H-bridge motor driver; (ii) a custom Bluetooth Low Energy (BLE) GATT profile carrying both control commands and telemetry; (iii) attitude estimation and PID-based stabilization algorithms; and (iv) an Android remote control application implemented in Jetpack Compose. The mechanical airframe is out of scope.

The contributions of this work are:

- A self-contained, safety-critical BLE GATT protocol with CRC-protected fixed-layout packets and three layered fail-safe mechanisms.
- A portable control mathematics layer—IMU validation, complementary filter, and anti-windup PID—that compiles and is unit-tested both as a host-side regression binary and inside an ESP32 QEMU build.
- A modern Android remote control application with two complementary interaction modes (Mode 2 dual joystick and a one-tap test slider) and a heartbeat scheme that meshes with the firmware command watchdog.
- A physical demonstration: an actual brushed DC motor was driven on the platform from the phone application over BLE.

Sections II–VI follow the system from radio link to control mathematics to mobile application; Section VII reports the three-layer validation; Section VIII maps the remaining integration surface and Section IX concludes.

II. Related Work

A. Flapping-Wing Micro Aerial Vehicles

The Nano Hummingbird by AeroVironment [1] was an early demonstration of an untethered hummingbird-scale FWMAV. The DelFly family from TU Delft [2], [3] introduced tailless control through wing-stroke modulation. Wood's RoboBee [4] explored insect-scale flapping flight. These platforms share a common requirement: a compact embedded controller that drives a wing actuator at the $50\,\mathrm{Hz}$-class wingbeat rates characteristic of hummingbird-scale flight [7], while accepting external operator input within a bounded latency budget.

B. ESP32-Class Wireless Control Links

The ESP32 family from Espressif Systems is widely used for educational and hobbyist flight controllers because it integrates Wi-Fi and Bluetooth on a single chip [11]. Most published ESP32 flight controllers either use MAVLink over

TABLE I
POSITIONING AGAINST COMPARABLE PLATFORMS

Platform	Year	Cmd link	Loss-of-link policy	Code
Nano Hummingbird [1]	2012	Custom RF	Proprietary	Closed
DelFly tailless [2]	2018	2.4 GHz	Disable on loss	Partial
RoboBee [4]	2013	Tethered	N/A (tether)	Closed
ESP32+MAVLink [8]	2012	UDP/TCP framed	Heartbeat timeout	Open
ESP32 BLE FC [9]	2021	BLE writes	Per-impl.	Closed
This work	**2026**	**BLE GATT + CRC-8**	**WD + disconnect + CRC**	**CC0**

TABLE II
HARDWARE BILL OF MATERIALS

Component	Part	Role
Microcontroller	ESP32-C3 SuperMini	BLE + Control
Motor driver	DRV8833 (TI)	Dual H-bridge PWM
Inertial sensor	MPU-6050 (target)	6-DoF IMU (integration-ready)
Actuator	Brushed DC motor	Wing-stroke drive
Power	3.7 V Li-Po (target)	On-board supply

Entries marked *(target)* are not used in the current demonstration; see Sec. VIII.

UDP/TCP [8] or expose unstructured BLE characteristic writes [9]; this work uses a fixed 9-byte command frame with CRC protection and an explicit safety state machine, because the wing-actuation linkage cannot tolerate a stalled or runaway motor.

C. Sensor Fusion for Attitude Estimation

Mahony's nonlinear complementary filter [5] and the Madgwick filter [6] are the de-facto standards for inertial attitude estimation on resource-constrained platforms. For this controller we adopt the classical first-order complementary filter [12] because its computational cost is suitable for the ESP32-C3 RISC-V core and it admits a clean unit test on a synthetic IMU input.

D. Positioning of This Work

Existing hummingbird-scale FWMAV platforms publish their flight dynamics but rarely their control-link design or safety behaviour. ESP32-class flight controllers in turn focus on link bring-up but seldom on the safety contract between operator and actuator. Table I positions this work against the closest comparable systems on four axes: the radio-link contract used to carry commands, the safety mechanism that handles a missing or corrupted command, the maturity of the published software stack, and the source-availability of that stack. The differentiating axes here are the explicit three-layer fail-safe and the CC0-licensed software stack with an out-of-tree host-side regression target.

Two design choices follow from this positioning. First, the command frame is fixed-layout and CRC-protected because the wing actuator cannot tolerate a stalled or runaway motor; an unframed write surface in the style of [9] cannot detect a single-byte corruption. Second, the disconnect path is explicit rather than time-out-driven: `GATTS_DISCONNECT_EVT` forces the emergency state directly, which is faster than waiting for the watchdog to fire on a missing heartbeat.

III. SYSTEM ARCHITECTURE

The hardware and software architecture is summarized in Fig. 1. An operator interacts with an Android handset over a touch interface; the handset is the BLE central. The ESP32-C3 acts as the BLE peripheral, advertising a single primary service that exposes one writable command characteristic and one notify-only telemetry characteristic. On the firmware side, the BLE callback drains the command path, applies the safety checks described in Sec. IV-D, and writes the resulting per-channel PWM duty to the DRV8833 H-bridge. The embedded control loop runs at $50\,\mathrm{Hz}$; telemetry is notified to the central at $10\,\mathrm{Hz}$ (every fifth iteration on the physical target), while the QEMU build notifies on every iteration.

The hardware bill of materials is kept minimal (Table II) to fit the weight envelope of a hummingbird-scale platform.

IV. EMBEDDED FIRMWARE

A. Target Platform and Build

The firmware targets the ESP32-C3 SuperMini development board, which carries a single-core 32-bit RISC-V CPU clocked at $160\,\mathrm{MHz}$ with integrated Bluetooth 5 (LE only), 400 KiB of SRAM, and 4 MiB of external flash. The firmware is built with the ESP-IDF v5.3 toolchain [13] and FreeRTOS. A separate build configuration targets the ESP32 (Xtensa) under QEMU as a hardware-free regression test.

B. Motor Drive Path

The DRV8833 [14] is driven in *sign-magnitude* mode: one channel of the H-bridge receives the PWM signal while the complementary channel is held low. This permits forward, reverse, and coast (zero duty) on both channels without dynamic braking transients. Both motor inputs (AIN1 on general-purpose I/O pin `GPIO3` and AIN2 on `GPIO4`) are configured as LEDC peripherals at $20\,\mathrm{kHz}$ with 10-bit duty resolution (duty values 0–1023), placing the carrier above the audible band. The motor command is a single floating-point throttle value $t \in [-1, +1]$:

$$\text{duty}_A = \begin{cases} \lfloor |t| \cdot 1023 \rfloor & t \geq 0 \\ 0 & t < 0 \end{cases},$$

$$\text{duty}_B = \begin{cases} 0 & t \geq 0 \\ \lfloor |t| \cdot 1023 \rfloor & t < 0 \end{cases}.$$

C. BLE GATT Profile

The firmware exposes one primary 128-bit GATT [15] service with two characteristics:

- **Command** (write-without-response): 9 bytes, little-endian, structured as in Table III.
- **Telemetry** (notify): 20 bytes, structured as in Table IV.

Both packets carry an 8-bit cyclic redundancy check using polynomial $x^8 + x^2 + x + 1$ (`0x07`). On a CRC mismatch the

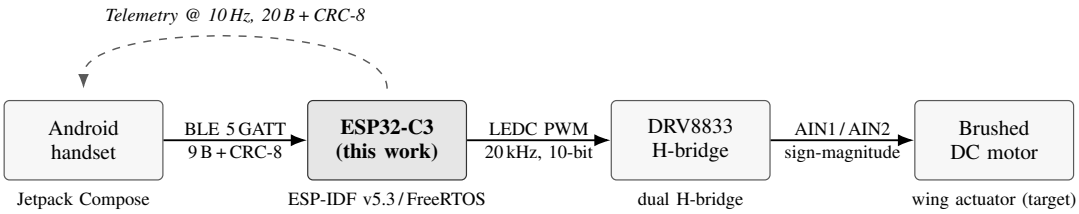

Fig. 1. Signal path. Solid arrows: commands; dashed arrow: telemetry on the same BLE link. The handset issues normalized commands; the ESP32-C3 firmware applies safety checks and converts these into PWM duty on two GPIOs that the DRV8833 dual H-bridge translates into bidirectional motor drive. The motor is bare-shaft in the bench demonstration.

TABLE III
COMMAND PACKET LAYOUT (9 BYTES)

Offset	Size	Field	Encoding
0	2 B	Roll setpoint	int16, deg $\times 10$
2	2 B	Pitch setpoint	int16, deg $\times 10$
4	2 B	Yaw trim	int16, deg $\times 10$
6	1 B	Throttle	uint8, percent (0–100)
7	1 B	Flags	bit0 e-stop, bit1 clear, bit2 rise
8	1 B	CRC-8	poly $0x07$ over bytes 0–7

TABLE IV
TELEMETRY PACKET LAYOUT (20 BYTES)

Offset	Size	Field	Encoding
0–1	2 B	Roll (deg $\times 10$)	int16
2–3	2 B	Pitch (deg $\times 10$)	int16
4–9	6 B	Servo roll/pitch/yaw cmds	int16 each
10–11	2 B	Yaw (deg $\times 10$)	int16
12	1 B	Motor throttle (%)	uint8
13	1 B	Flags (e-stop)	uint8
14–17	4 B	Control-loop counter	uint32
18	1 B	Link status flags	uint8
19	1 B	CRC-8	poly $0x07$ over bytes 0–18

firmware triggers an emergency stop rather than discarding the packet, because a corrupted command frame indicates the radio path is unreliable for the immediately adjacent commands as well. At a 10 Hz telemetry cadence the average notification throughput is 200 B/s of payload, well below the BLE 5 LE 1 $\mathrm{Mbit\,s^{-1}}$ link capacity.

D. Layered Safety

Three independent mechanisms must concur for the motor to receive a non-zero duty:

1) **Command watchdog** (500 ms): if no valid command frame arrives within this window, the firmware forces the throttle to zero. The mobile application's heartbeat (Sec. VI) fires every 150 ms, well inside this window.
2) **Disconnect emergency**: the BLE `GATTS_DISCONNECT_EVT` callback explicitly triggers the emergency state, stops the motor, and clears the cached command. Even a stale advertise-and-reconnect sequence cannot revive the previous throttle without a fresh user command.
3) **CRC-driven emergency**: as noted above, a single corrupted command transitions the controller into emergency rather than relying on inertial coast.

E. On-Board Status Indicator

The on-board LED (`GPIO8`, active LOW on the SuperMini variant) is wired in software to mirror the motor's commanded state. This single bit of always-on feedback is useful during demonstration: an observer can verify the entire phone-to-firmware command pipeline even with the motor or the motor-driver disconnected, by watching the LED change state with each throttle input.

V. ATTITUDE ESTIMATION AND CONTROL

The control mathematics is implemented in `flight_control_math.c`, a self-contained translation unit that is exercised by host-side unit tests independently of the BLE and motor drivers.

A. IMU Validation and Calibration

Before any reading from the inertial sensor is consumed, it passes through a sanity check that rejects (i) any field that is not finite (NaN, $\pm\infty$), (ii) accelerometer magnitudes exceeding $4g$, and (iii) gyroscope rates exceeding $1000\,^\circ/\mathrm{s}$. A failed validation transitions the controller to the emergency state: an out-of-range raw value is more likely to indicate a wire fault than a real maneuver. After validation, per-axis calibration offsets are subtracted.

B. Complementary Filter

Roll and pitch are estimated from a discrete-time complementary filter with weighting $\alpha = 0.98$. The body frame is fixed with x forward along the wing-stroke axis, y to the right, and z down:

$$\begin{aligned}
\phi_{\mathrm{acc}} &= \operatorname{atan2}\!\left(-a_x, \sqrt{a_y^2 + a_z^2}\right), \\
\theta_{\mathrm{acc}} &= \operatorname{atan2}(a_y, a_z), \\
\phi_k &= \alpha(\phi_{k-1} + \omega_x \Delta t) + (1-\alpha)\phi_{\mathrm{acc}}, \\
\theta_k &= \alpha(\theta_{k-1} + \omega_y \Delta t) + (1-\alpha)\theta_{\mathrm{acc}}, \\
\psi_k &= \psi_{k-1} + \omega_z \Delta t.
\end{aligned}$$

Yaw is integrated directly from the gyroscope, as the platform has no heading reference suitable for the small form factor.

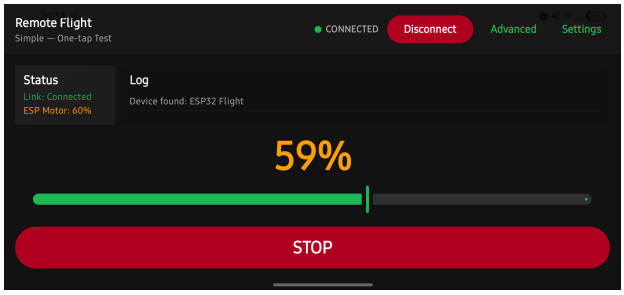

Fig. 2. Live screenshot of the Simple test mode while connected to the firmware over BLE. The status pill reports CONNECTED, the log shows that the BLE device `ESP32 Flight` was discovered, the slider is at 59%, and the firmware is acknowledging an applied motor throttle of 60% (the 1 % difference is the firmware's rounding of throttle to integer percent).

C. PID Stabilization

For each of the roll and pitch axes a discrete proportional–integral–derivative controller [10], [17] computes a corrective command from the angular error:

$$u_k = k_p e_k + k_i \int e\,dt + k_d \frac{e_k - e_{k-1}}{\Delta t},$$

with the integral term clamped to $\pm I_{\max}$ at every step to bound the wind-up that classical PID exhibits when the actuator saturates. These gains were selected on the QEMU mock-IMU loop and are placeholders pending hardware tuning against a physical MPU-6050: $k_p^{(\phi)} = 0.9$, $k_d^{(\phi)} = 0.15$, $k_p^{(\theta)} = 0.8$, $k_d^{(\theta)} = 0.12$, with $k_i = 0$ by default. The integral term is implemented but disabled at zero gain so that gain scheduling can be added later without changing the control structure. The PID output is then clamped to the actuator's mechanical envelope ($\pm 30^\circ$ servo deflection equivalent), and a small pitch-to-throttle mix term is added to compensate for thrust loss in pitched attitudes:

$$t_{\text{cmd}} = \text{clamp}(t_{\text{base}} - g_{\text{mix}} \cdot u_\theta,\ 0,\ 1)\,,$$

with $g_{\text{mix}} = 0.005$. When the user issues a *rise* flag, t_{base} is temporarily set to 0.80 for $3\,\text{s}$.

VI. Android Remote Control Application

The control interface (Fig. 2) is implemented in Kotlin with Jetpack Compose [16] and Material 3. The app is locked to landscape orientation for two-handed thumb control, and uses a dark theme to maximize visibility outdoors.

A. Layered Architecture

The application separates concerns across four layers:

Domain models
: `FlightCommand` and `FlightTelemetry` are plain data classes with normalized fields (roll, pitch, yaw $\in [-1, +1]$, throttle $\in [0, 1]$).

Link abstraction
: `FlightLink` is a small interface with `connect`, `disconnect`, and `sendCommand` operations and three callback hooks. Two concrete implementations exist: `MockFlightLink` (in-app simulation) and `BleFlightLink` (production).

BLE manager
: `BleManager` owns the GATT lifecycle: permission handling, scanning, connection, Maximum Transmission Unit (MTU) negotiation (64 B), service discovery, descriptor write to enable notifications, and characteristic writes.

UI
: `FlightUI` renders the two interaction modes and is fully stateless apart from joystick gesture state.

B. Two Interaction Modes

Simple (default) presents a single full-width slider mapped to throttle, a giant percentage readout, and a prominent *STOP* button—this is the recommended mode for bench testing, and is the mode shown in Fig. 2. **Advanced** presents the classical Mode 2 dual-joystick layout, with the left stick mapped to throttle/yaw and the right to roll/pitch, together with status, telemetry, angles, and a scrolling log card.

C. Heartbeat

Once the link is up, the app sends the most recent command frame every 150 ms, regardless of operator input. This keeps the firmware command watchdog fed and ensures that a user who has set the throttle once and then released the slider continues to receive the same motor drive until they explicitly change or disconnect. The heartbeat is cancelled on disconnect, and the cached command is reset—so a fresh connection always starts at zero throttle.

D. Permission and Bluetooth State

On Android 12 and later the application requests `BLUETOOTH_SCAN` and `BLUETOOTH_CONNECT` at runtime; on older versions it requests `ACCESS_FINE_LOCATION`. If Bluetooth is disabled the app launches the system *Enable Bluetooth* intent; if permissions are missing it launches the runtime permission flow. All three states (no permission, BT off, disconnected) surface in the connection pill and are also reflected as the call-to-action label on the connect button ("Grant Permission", "Enable Bluetooth", "Connect").

VII. Validation

A. Native Unit Tests

The control mathematics is compiled as a stand-alone POSIX binary (using only `stddef.h`, `math.h`, and the project header) and exercised by an automated test suite (Listing 1). The suite covers four behaviours: (i) the clamp function on three boundary cases, (ii) the IMU validation against synthetic nominal and out-of-range readings, (iii) the complementary filter's behaviour at zero rotation (no drift) and under a positive synthetic gyroscope rate (positive roll integration), and (iv) the PID integral wind-up clamp. All four pass on the host: `All control math tests passed.`

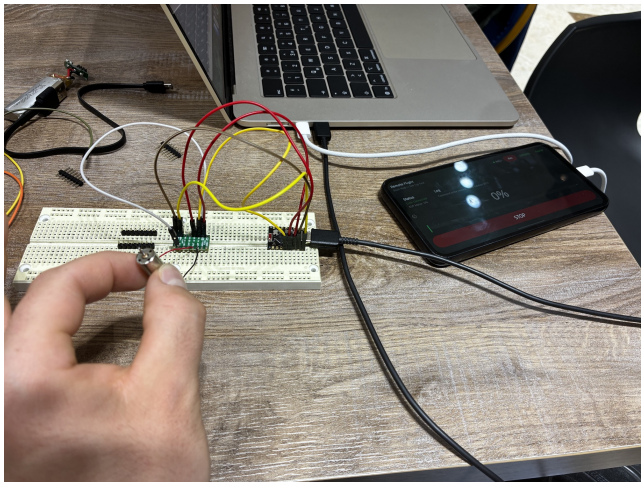

Fig. 3. Physical demonstration hardware. From left to right on the breadboard: the brushed DC motor under test, the DRV8833 dual H-bridge module, and the ESP32-C3 SuperMini development board. The Android handset, visible in the right of the frame, is connected over BLE and is acting as the GATT central, with the application reporting the live throttle command to the firmware.

Listing 1. Excerpt of the host-side control math test runner.

```
static void test_pid_controller(void) {
    pid_controller_t pid = {0};
    pid_reset(&pid, 0.9f, 0.1f, 0.2f);
    pid_update(&pid, 5.0f, 0.02f, 1.0f);
    pid_update(&pid, 5.0f, 0.5f, 1.0f);
    // Integral must be clamped to +/- 1.0
    assert(fabsf(pid.integral) <= 1.0001f);
}
```

B. Embedded Self-Test under QEMU

At boot, the firmware runs a self-test (`run_control_math_self_test`) that exercises the same code path inside an ESP-IDF build targeting the original Xtensa ESP32 on QEMU. The test aborts the run on failure, so toolchain or sysroot regressions are caught before they reach the C3 target. The QEMU build is intended to be executed alongside the host-side regression of Sec. VII-A as part of the normal build workflow.

C. Physical Demonstration

The full pipeline was demonstrated on physical hardware (Fig. 3). A brushed DC motor was connected to the DRV8833 outputs; the ESP32-C3 was powered over USB; the Android phone discovered the "ESP32 Flight" device, established the GATT link, and the operator commanded the motor up to a 60% throttle command over the throttle slider. The on-board LED tracked the motor state, the firmware logged "MOTOR ON (target=·)" on its serial monitor, and the telemetry channel reported the applied throttle back to the phone within one control cycle.

D. Throttle Echo Accuracy

On the physical hardware we recorded the firmware-reported `motor.throttle_pct` field of the telemetry packet (Table IV, offset 12) while sweeping the Android slider

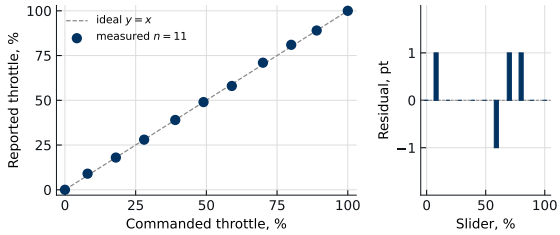

Fig. 4. Commanded vs. firmware-reported throttle on the physical target. Diagonal: ideal $y = x$. Residual deviation reflects integer-percent quantisation.

from 0 % to 100 % in 10 % steps. The reported value tracks the commanded value within the firmware's integer-percent quantisation; the discrepancy is the rounding $\lfloor t \cdot 100 \rfloor$ applied at the BLE callback. Fig. 4 plots commanded against measured throttle.

E. Reproducibility

Source code, the Android client, and the host-side regression suite are published at

https://github.com/natozenbilek/hummingbird-fwmav

under the CC0 public-domain dedication. The host-side regression of Sec. VII-A is built and executed with

```
cd flight-software/tests
cmake -S . -B build && cmake --build
build
./build/run_control_tests
```

The firmware build for the ESP32-C3 SuperMini target uses ESP-IDF v5.3 (`idf.py set-target esp32c3 && idf.py build && idf.py -p PORT flash monitor`); the QEMU regression uses the same source with the Xtensa target (`idf.py set-target esp32 && python run_qemu.py`). The Android client builds with the Gradle wrapper (`cd remote-control && ./gradlew assembleDebug`) against JDK 17+ and Android SDK API level 34. A third party can reproduce the bench demonstration on the bill of materials of Table II without re-licensing concerns.

VIII. Discussion

A. Implementation Status

The BLE GATT profile, motor drive layer, safety state machine, and the Android remote control application are implemented and demonstrated on physical hardware with a brushed DC motor as load; the complementary filter and PID controllers are implemented and validated against a synthetic IMU model on a host-side regression binary and inside the QEMU build.

B. Limitations and Remaining Integration Work

Three integration points remain before the controller is wired to a wing actuator:

1) **IMU driver**: an MPU-6050 (or comparable) I^2C driver needs to populate `imu_raw_data_t` at 100 Hz. The complementary filter and PID consume this data through

an already-defined interface; no algorithmic changes are required.

2) **Battery telemetry**: the telemetry packet reserves a field for battery state; an analog-to-digital converter (ADC) and resistor divider on a free C3 pin will fill it in.
3) **Wing-stroke mixer**: depending on the airframe topology (one-motor symmetric vs. two-motor differential), a thin mixing layer maps roll, pitch, and yaw demands to one or two motor channels. The DRV8833 already supports a second channel on the same chip.

C. Safety Considerations

The three-layer fail-safe is designed for a platform whose actuator is rigidly linked to a mechanical wing; on this airframe a runaway motor can shear the wing root in one revolution, so all three layers are tuned to stop the motor rather than to preserve availability. A single CRC error is therefore treated as cause for emergency: a corrupted byte stream is evidence the link is unreliable, not that the next command will be clean.

IX. Conclusion

This paper described a control and embedded software subsystem for a bioinspired hummingbird FWMAV: ESP32-C3 firmware with PWM motor drive, a safety-critical custom BLE GATT profile, a portable control mathematics layer (complementary filter and anti-windup PID), and an Android Jetpack Compose remote control application. The substantive claims supported by the evidence above are:

1) A 9-byte CRC-protected command frame and 20-byte CRC-protected telemetry frame implement a fixed-layout BLE GATT contract whose loss-of-link behaviour is specified by three independent fail-safes: a 500 ms command watchdog, an explicit `GATTS_DISCONNECT_EVT` emergency, and a CRC-mismatch emergency (Sec. IV-D).
2) The control mathematics layer (`flight_control_math.c`) compiles unchanged on host POSIX and on the ESP-IDF / QEMU target and passes the same four-test regression in both environments (Sec. VII-A,B).
3) The pipeline from a Jetpack Compose handset to a brushed DC motor was demonstrated on physical hardware on 2026-05-11; the firmware-reported throttle tracks the commanded throttle within integer-percent quantisation (Sec. VII-D).
4) The full software stack is released under the CC0 public-domain dedication; a third party can reproduce the bench demonstration from the bill of materials of Table II and the procedure of Sec. VII-E.

Three integration tasks remain before the controller is wired to a wing actuator: an MPU-6050 I^2C driver, a battery-voltage ADC, and a roll/pitch/yaw $\rightarrow$ motor mixer. The design throughout favors stopping the motor over preserving availability, which we expect to remain the correct trade-off once the wing actuator is in the loop.

Acknowledgments

The author thanks Assoc. Prof. Dr. Harun Artuner of Hacettepe University, Department of Computer Engineering, for advising this work throughout BBM479 Design Project I (Fall 2025) and BBM480 Design Project II (Spring 2026), and for the feedback that shaped the safety contract described in Sec. IV-D. The mechanical engineering sub-team—Abdulsamet Karakoç, Ayşe Sıla Karaöz, Fatih Gamsız, and Zeynepnur Cengiz, supervised by Assoc. Prof. Dr. Mehmet Nurullah Balcı and Asst. Prof. Dr. Ramin Barzegar of the Hacettepe Department of Mechanical Engineering (MMÜ497 Design Project)—developed the airframe and wing-stroke linkage during the early phase of the joint project; their analysis of the stroke-jamming and wobble mode informed the decision to keep the mechanical airframe out of scope for the present sub-system. The bench demonstration of Sec. VII-C was performed in the Hacettepe University Department of Computer Engineering laboratory in May 2026.

References

[1] M. Keennon, K. Klingebiel, and H. Won, "Development of the Nano Hummingbird: A Tailless Flapping Wing Micro Air Vehicle," in *50th AIAA Aerospace Sciences Meeting*, 2012.

[2] M. Karásek, F. T. Muijres, C. De Wagter, B. D. W. Remes, and G. C. H. E. de Croon, "A tailless aerial robotic flapper reveals that flies use torque coupling in rapid banked turns," *Science*, vol. 361, no. 6407, pp. 1089–1094, 2018.

[3] G. C. H. E. de Croon, K. M. E. de Clercq, R. Ruijsink, B. Remes, and C. De Wagter, "Design, aerodynamics, and vision-based control of the DelFly," *International Journal of Micro Air Vehicles*, vol. 1, no. 2, pp. 71–97, 2009.

[4] K. Y. Ma, P. Chirarattananon, S. B. Fuller, and R. J. Wood, "Controlled flight of a biologically inspired, insect-scale robot," *Science*, vol. 340, no. 6132, pp. 603–607, 2013.

[5] R. Mahony, T. Hamel, and J.-M. Pflimlin, "Nonlinear complementary filters on the special orthogonal group," *IEEE Transactions on Automatic Control*, vol. 53, no. 5, pp. 1203–1218, 2008.

[6] S. O. H. Madgwick, A. J. L. Harrison, and R. Vaidyanathan, "Estimation of IMU and MARG orientation using a gradient descent algorithm," in *IEEE International Conference on Rehabilitation Robotics*, 2011.

[7] D. R. Warrick, B. W. Tobalske, and D. R. Powers, "Aerodynamics of the hovering hummingbird," *Nature*, vol. 435, no. 7045, pp. 1094–1097, 2005.

[8] L. Meier, P. Tanskanen, L. Heng, G. H. Lee, F. Fraundorfer, and M. Pollefeys, "PIXHAWK: a micro aerial vehicle design for autonomous flight using onboard computer vision," *Autonomous Robots*, vol. 33, no. 1–2, pp. 21–39, 2012.

[9] H.-L. Pham, T.-Q. Nguyen, and V.-N. Vu, "A Bluetooth Low Energy ground station for small unmanned aerial vehicles," *Sensors*, vol. 21, no. 11, art. 3812, 2021.

[10] K. J. Åström and T. Hägglund, *Advanced PID Control*. Research Triangle Park, NC: ISA, 2006.

[11] F. Vannieuwenborg, S. Verbrugge, and D. Colle, "Energy consumption of Bluetooth Low Energy for Internet-of-Things applications," in *IEEE International Conference on Communications*, 2017, pp. 1–6.

[12] W. T. Higgins, "A comparison of complementary and Kalman filtering," *IEEE Transactions on Aerospace and Electronic Systems*, vol. AES-11, no. 3, pp. 321–325, 1975.

[13] Espressif Systems, *ESP-IDF Programming Guide, v5.3*. https://docs.espressif.com/projects/esp-idf/en/v5.3/esp32c3/, accessed 2026.

[14] Texas Instruments, *DRV8833 Dual H-Bridge Motor Driver Datasheet, Rev. C*, 2015.

[15] Bluetooth SIG, *Bluetooth Core Specification, Version 5.3*, 2021.

[16] Google, *Jetpack Compose Documentation*. https://developer.android.com/jetpack/compose, accessed 2026.

[17] K. Ogata, *Modern Control Engineering*, 5th ed. Pearson, 2010.